

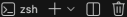
count with array

Array	{ }	{1,4,3,1,2,0,4,2}
Name of array	frequency	member
Length	5	8
Value	frequency[i]	member[i]<frequency.size()
for loop	frequency output	meaning
for(size_t i=0;i<member.size();i++){ frequency[i]++;}	{1,1,1,1,1}	Every element ‘zero’ add ‘1’.
for(size_t i=0;i<member.size();i++){ frequency[mamber[i]]++;}	{1,2,2,1,2}	To count appearance of every element of array ‘member’ at array ’frequency’.
for(int &i:member){frequency[i]++}	{1,2,2,1,2}	Every element value of array ‘member’ refer to the index of the array ‘frequency’ and the index add ‘1’.

```
1 #include <array>
2 #include <iostream>
3 using namespace std;
4
5 int main(){
6     const size_t arrayASize=5;
7     const size_t arrayBSize=8;//longer than arrayASize
8     const size_t arrayCSize=3;//shorter than arrayASize
9     const size_t arrayDSize=5;//value is bigger than arrayASize
10
11     array<int,arrayASize>arrayA1={};
12     array<int,arrayASize>arrayA2={};
13     //array<int,arrayASize>arrayA3_1={};
14     array<int,arrayASize>arrayA1_2={};
15     array<int,arrayASize>arrayA2_2={};
16     //array<int,arrayASize>arrayA3_2={};
17     array<int,arrayBSize>arrayB1={1,4,3,1,2,0,4,2};
18     array<int,arrayBSize>arrayB2={1,4,3,1,2,0,4,2};
19     array<int,arrayCSize>arrayC1={1,4,3};
20     array<int,arrayCSize>arrayC2={1,4,3};
21     //array<int,arrayDSize>arrayD1={1,8,3,11,8};
22     //array<int,arrayDSize>arrayD2={1,8,3,11,8};
23
24     for(size_t i=0;i<arrayB1.size();i++){arrayA1_1[arrayB1[i]]++;}
25     for(size_t i=0;i<arrayC1.size();i++){arrayA2_1[arrayC1[i]]++;}
26     //for(size_t i=0;i<arrayD1.size();i++){arrayA3[arrayD1[i]]++;}
27
28     cout<<"arrayA1_1: ";
29     for(int &i:arrayA1_1) cout<<i<<" ";
30     cout<<endl;cout<<"arrayA2_1: ";
31     for(int &i:arrayA2_1) cout<<i<<" ";
32     cout<<endl;
33     //for(int &i:arrayA3) cout<<i<<" ";
34 }
```

example .cpp

```
34
35
36     for(int &i:arrayB2)arrayA1_2[i]++;
37     for(int &i:arrayC2)arrayA2_2[i]++;
38     //for(int &i:arrayD2)arrayA3_2[i]++;
39
40     cout<<"arrayA1_2: ";
41     for(int &i:arrayA1_2) cout<<i<<" ";
42     cout<<endl;cout<<"arrayA2_2: ";
43     for(int &i:arrayA2_2) cout<<i<<" ";
44     cout<<endl;
45     //for(int &i:arrayA3) cout<<i<<" ";
46
47     return 0;
48 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 

(base) pengengei@pengengeideMacBook-Pro ~ % cd Desktop/c++/0424
编译
(base) pengengei@pengengeideMacBook-Pro 0424编译 % g++ array0-2
.cpp -o program
(base) pengengei@pengengeideMacBook-Pro 0424编译 % ./program
arrayA1_1: 1 2 2 1 2
arrayA2_1: 0 1 0 1 1
arrayA1_2: 1 2 2 1 2
arrayA2_2: 0 1 0 1 1
(base) pengengei@pengengeideMacBook-Pro 0424编译 % █

example .cpp

Built-in array

```
int main(int argc,char* argv[]){}

char* argv[ ]
={ &argv[0], &string1[0], &string1[0],...}
={ argv, &char1-1, &char2-1,...}
```

Syntax	Difinition	Sort	Meaning
char*	A "pointer" points to the first character of a string.	type	Begining "address" of a string.
argv[]	Including elements, each element is a pointer.	array	An array "made of address" of strings' first character.

```
argv=&argv[0]:
```

The address of the first element of the subordinate array in another hand, &array[0] is assigned to The name of an array.

value	content type stored		content inputed in terminal	content type stored in memory
argv[0]	char1*	array's begining memory addreass	Name of "main" function, not wit .cpp	char
argv[1]	char2*	The first string's begining memory address.	The first command	char
argv[2]	char*3	The second string 's begining memory address.	The second command	char

If command is number:

```
Standard Library:  scanf
tool:              atof,atoi
deal:              content inputed in terminal
method:            translate with ASCII
```

const usage	address pointed to	content pointed to in memory	example
int *countPtr;	can be change	can be change	void displayArray(int *array, size_t size);
const int *countPtr;	can be change	can't ba change	void displayArray(const int *array, size_t size);
int *const countPtr = &x;	can't ba change	can be change	void displayArray(int *const array, size_t size);
const int *const countPtr = &x;	can't ba change	can't ba change	void printArray(const int *const array, size_t size);

int	content type	meaning	*	ex
int *n	address	claim a pointer "n" to store an address of integral	turn the type of "n" to be a pointer	ox61ff08
a	integral	integral		7
&a	address	memory address of integral "a"		ox61ff08
n	address	a pointer stores memory address of "a"		ox61ff08
*n	integral	"a" : point to content of "n"	point to the address stored by "n" to get content inside it	7

```
void fun1(int *n)
fun1(&a)
```

	content type	meaning	*	ex
int *n	address	claim a pointer "n" to store an address of integral	turn the type of "n" to be a pointer	ox61ff08
a	array-->address	name of an array and turn to be &a[0]		{ x1 , x2 , x3 , ... }-->x1
&a[i]	address	memory address of intgral a[i]		ox61ff08
n	address	a pointer stores memory address of &a[0] or a[]		ox61ff08
*n	intrgral (a[0])	"a" : point to content of "n"	point to the address stored by "n" to get the content inside it	x1

put &a[0]: &a[...]:memorial address of the hole array,not a integral.

```
void fun1(int *n);
fun1(a);
```

```
voif fun1(int *n);
fun1(&a[0]);
```

array	content type	meaning	*	ex
int *n[]	address	claim pointers "n[i]" to store addresses of integrals	turn the type of "n[i]" to be a pointer	ox61ff08
a	intgral	intgral to be an alement		7
&a	address	memory address of intgral "a"		ox61ff08
n[i]	address	a pointer stores memory address of &n[i]		ox61ff08
*n[i]	integral	"a" : point to content of "n[i]"	point to the address stored by "n[i]" to get the content inside	7

	content type	meaning	*	ex
char *argv[]	array for address	claim pointers "argv[i]" to store addresses of chars	turn the type of "argv[i]" to be a pointer	ox61ff08
a	the string	name of function "main" or another command from terminal		./program
&a	address	memory address of any command from teminal		ox61ff08
argv[i]	address	an pointer stores memory address of any conmmand		ox61ff08
*argv[i]	the first char of string	"a" : point to content of "argv[i]"	point to the address stored by "argv[i]" to get content inside it	.

```
code editor:
int main(int argc,char *argv[]){
    return 0;
}
```

```
terminal:
./program 2.0 0.5
```

```
command:./program,2.0,0.5
type:string
```

Offset notation

```
A parameter "a" 's type is unknown.
Its address: &a
Claim its pointer: void *n
```

```
"void *n" equals to "&a" equals to "n"
```

```
If there are two claim:
int *p=&b;
void *n;
n=p;
p!=n;
int *p=reinterpret_cast<int*>(n);
```

```
The knowable can be assigned to the unknown.
The unknown can;t be assigned to the knowable.
```

```
An array "a[n]"
```

address	calulation	equals to	meaning	unit	content(value)	meaning
a	a+number	&[0+number]	&a[0]+number*4	1 element / 4 bytes	*(a+number)	a[number]

```
example:
```

```
ptr=&a[0]
*(ptr+3)=a[3]=ptr[3] :*(&a[0]+3*4)=*(&a[3])=*(address+3*4)=*(a+3)
*ptr+3=&a[0]+3 :a[0]+3
```

value	write method	operation
a[3]	a+3	legal
a[0]=a[3]	a+=3	illegal

Address

syntax	type	identity	segments
"Hello"	address	string	Constant Segment
'H'	character	value	Stack

example syntax	segment	fixed member	Accessing variable across file	variable
new int[a].....delete pointer	Heap			yes
int a	Global Segment		external linkge	yes
static int a	Static Segment		internal linkage	yes
const int *a=7	Constant Data	type		no
const int a=7	Text Segment	value		no
void fun(int a){...}	Text Segment			yes
content in fun(int a){ }	Stack			yes

in Constant Segment:
address: 0x4...=="Hello"
content:'H','e','l','l','o','\0'

in Stack:
copy the content from address "Hello" in Constant Segment to a[]in Stack:
char a[]="Hello";
the elements in a[] is:
char a[]={'H','e','l','l','o','\0'};

so a[0]='H',a[1]=e... can be edited
but "Hello" still exists in Const Segment,that:

```
char* ptr="Hello";  
ptr[0]='h';
```

is illegal

```
&a[0]==a;  
a!=&('H');  
char c='H';  
a==&c;
```

```
int a[7]={0,2,4,6,8,10,12};
```

```
int *ptr=&a[0];
```

that :

```
int *ptr=a
```

```
int *ptr==&a[0]
```

```
ptr==&a[0]==a Notice:ptr is a variable value but a is a name of an array,means ptr++ is legal but a++ is illegal
```

```
*ptr==a[0]==ptr[0]
```

```
so a[1]==*(&a[0]+1)==*(a+1)==*(ptr+1)==ptr[1]
```

```
int *ptr=&a[2];
```

that:

```
int *ptr==a+2
```

```
so ptr==&a[2]==&a[0]+2==a+2
```

```
*ptr==a[2]==ptr[0]
```

```
so ptr[1]==*(ptr+1)==*(a+2+1)=a[3]
```

if a class Object ,which is the object,and has a member:

```
class Object{  
    int memberx;  
    void member();  
}
```

calling m's method:

```
Object obj;
```

```
obj.mx
```

```
obj.my
```

set a pointer:

```
char* ptr=&obj;
```

```
ptr->mx==obj.mx==(*ptr).mx==(&obj)->mx
```